If the constructor arguments include a dependency from the container, the factory provides access to the service, so you can resolve the dependency. The following example shows how this works if the `SimpleCityService` takes a dependency on an implementation of `IMyService` as well as `args`. You use the `IServiceProvider GetService` method to resolve the dependency. We will look at other ways to access services directly from the service provider at the end of the chapter:

```
builder.Services.AddTransient(provider =>
    new SimpleCityService(args, provider.GetService<IMyService>())
);
```

The factory option is preferred because it hands responsibility for newing up, or *activating*, the service to the service container. And if the container is responsible for service activation, it also takes responsibility for service disposal. There is an alternative method that applies to singleton services, which involves passing in the constructed service:

```
builder.Services.AddSingleton<IMyService>(new MyService(args));
```

When you register a service using this approach, you must take responsibility for its disposal too. The same is true if you pass in the constructed service using the implementation-only option:

```
builder.Services.AddSingleton(new MyService(args));
```

## *7.2.5 Registering multiple implementations*

It is possible to register multiple implementations of a service by repeating the relevant `Add[LIFETIME]` method with the same service type but a different implementation: